

Deployment Best Practices Report Script

Step 1: Introduction (Setting the Stage)

"Good day everyone. For this part of the report, I will walk you through the deployment process of our Multi-Branch Inventory Management System. The goal of deployment is to take our application from a local development environment (our laptops) and put it on a live server so that users across different branches can access it securely over the internet. We focused on five key areas to ensure a smooth transition to production."

Step 2: Preparing the Application for Deployment

"The first step is preparing the application. In development, our Django REST Framework (DRF) and FastAPI backends run in a relaxed mode, but in production, we need to enforce strict rules:

1. Disabling Debug Mode: We turn off `DEBUG = True` in Django. If left on, it could expose sensitive backend code if an error occurs.
2. Setting Allowed Hosts: We configure `ALLOWED_HOSTS` to ensure our backend only accepts requests from our official domain names, preventing unauthorized access.
3. Static & Media Files: We set up proper handling for static files (like CSS/JS) and media files (like product images) using libraries like `WhiteNoise`, ensuring they load quickly and correctly on the live server.
4. Dependency Management: We freeze all our required libraries into a `requirements.txt` file so the server knows exactly what to install."

Step 3: Environment Variables and Configuration

"The next critical step is securing our configuration using Environment Variables. Instead of hardcoding sensitive information like database passwords or secret keys directly into our code, we use environment variables. This means our code simply asks the server, 'What is the secret key?' and the server provides it securely. This ensures that even if our source code is viewed by someone else, our database credentials, API keys, and security tokens remain completely hidden and safe."

Step 4: Database Setup (PostgreSQL)

"For our database, we transition from the default local database (like SQLite) to PostgreSQL. PostgreSQL is a powerful, enterprise-grade relational database. We chose this because:

Deployment Best Practices Report Script

1. It handles concurrent users efficiently, which is crucial since multiple branches will be accessing the system simultaneously.
2. It offers strict data integrity.
3. To connect to it, we use a Database URL (provided by our hosting platform) which we inject into our system via the environment variables we discussed earlier. We then run our 'migrations' on the live database to set up our tables perfectly."

Step 5: Using Platforms like Render

"To actually host the application, we utilize modern cloud platforms like Render. We connected our code repository (GitHub) directly to Render. The process works like this:

1. Web Services: We create a Web Service for our Django/FastAPI backend and our React frontend.
2. Managed Database: We provisioned a managed PostgreSQL database directly within Render.
3. Continuous Deployment (CI/CD): Render gives us automated deployments. This means whenever we push an update or bug fix to our main GitHub repository, Render automatically detects the changes, rebuilds the application, and deploys the new version without any downtime."

Step 6: Monitoring and Maintaining Deployed Applications

"Finally, deployment is not a one-time task; it requires ongoing monitoring. Once live, we maintain the system by:

1. Checking Server Logs: We actively monitor the application logs provided by Render. This allows us to track incoming requests, monitor performance, and immediately spot if an error occurs.
2. Database Backups: We ensure our PostgreSQL database has automated daily backups to prevent any data loss in case of a disaster.
3. Uptime Monitoring: We keep an eye on the server's memory and CPU usage to ensure the system doesn't crash during peak hours when multiple branches are doing inventory."

Summary / Conclusion

"To summarize, our deployment strategy ensures that the Multi-Branch Inventory Management System is highly secure through environment variables, robustly supported by a PostgreSQL

Deployment Best Practices Report Script

database, seamlessly hosted on Render, and constantly monitored for peak performance. This guarantees a reliable experience for all branches using the system. Thank you."